SemPointer: Adaptive Lexical Routing for Token-Efficient Long-Context Question Answering

Abstract. Long-context question answering trades evidence coverage against the tokens and time spent on every query. SemPointer stores text in blocks, retains short address pointers in the active prompt, and resolves selected blocks for generation. Its first version fused lexical, dense, and model-attention rankings, but the attention component hurt retrieval on the tested tasks. We replace that selector with a fast lexical router that protects two BM25 blocks and expands to four when query-term coverage or ranking ambiguity indicates greater evidence demand. Across 180 RULER examples, the new design improves the earlier SemPointer score by 16.6 percentage points (paired 95% confidence interval [11.0, 22.6]). Against BM25 retrieval with four blocks in the same run, it scores 62.1% versus 62.3%, uses 8.9% fewer active tokens, and reduces mean per-query answer latency by 15.7% (ingestion excluded). On the selected LongBench and LoCoMo subsets, score differences from BM25-4 are small and inconclusive, while mean token and answer-latency costs are lower. The result is a practical context-budget mechanism on this single-model evaluation, not a claim of accuracy superiority or official benchmark standing. A held-out routing evaluation and pointer-specific control remain necessary.

Keywords: Long-context language models · adaptive retrieval · semantic pointers · context efficiency

1 Introduction

Repeated questions about a large document or conversation need different evidence from the same stored history. Feeding every token to the language model gives broad access but repeats a large prefill. A retrieval system reduces that cost by sending selected passages. The central engineering question is how much evidence to expose for each query without paying for a fixed, generous passage count every time.

SemPointer attaches a stable, short pointer to each block of stored text. A query-time selector chooses block identifiers, and the resolver replaces their pointers with full text while leaving the others as compact entries. The generation model sees this mixed prompt. The pointer is an explicit address in a registry, not a learned latent reconstruction code. Its value should be evaluated together with the selector and the cost of presenting unresolved pointers; a shorter prompt alone does not establish a better answering system.

Our first implementation (the earlier version) combined BM25, dense similarity, and an attention pass over the pointer index. On the completed RULER evaluation, its default system scored 45.6% against 62.3% for four-block BM25 retrieval. BM25-only selection scored substantially better in a separate ablation of that version. That diagnosis motivated the present design: a lexical-first router which

spends its normal budget on two blocks and expands only when a query appears undersupported or ambiguous. It avoids the extra model-attention selection pass and does not load a dense embedding model in the measured configuration.

This paper reports the completed runs, including all 180 RULER, 600 LongBench, and 497 LoCoMo examples in the specified subsets. Our central finding is narrow but useful: relative to fixed four-block BM25 retrieval, the router obtains similar observed answer scores with lower mean active-token count and lower same-run latency across these three collections. It does not establish that pointer text is responsible for the gain, and it remains below full context on RULER.

2 Related Work and Scope

BM25 and dense retrieval select external evidence for retrieval-augmented generation [14, 6, 13]. Reciprocal-rank fusion combines rankings, while landmark-style and learned compressed representations address different forms of long-context access [3, 10, 11, 2]. SemPointer uses an ordinary text registry with block identifiers and exact text resolution. The contribution here is an adaptive budget policy over that registry, not a new retrieval score or an end-to-end trained memory representation.

Cache compression and reusable KV states address related compute costs [7, 8]. The present results use text resolution only. They neither inherit the cached-KV measurements of the earlier version nor demonstrate a KV advantage for the new router. We compare SemPointer with BM25-2 and BM25-4 where available, and with the earlier version and full context on shared example IDs when earlier runs permit. Same-run comparisons carry the strongest timing interpretation.

3 Method

3.1 Address layer and prompt construction

Let the stored history be divided into N text blocks X_i of at most $B = 512$ target-model tokens. A registry contains triples (i, P_i, X_i), where i is a stable identifier and P_i is a short, readable pointer capped at $k = 16$ tokens. As in the earlier version, pointer terms favor block-distinctive words, numbers, and entity-like strings. Selected blocks appear as full text labeled by identifier; unselected blocks remain as pointers. For selected set $S(q)$ and question q, the idealized prompt length is

$$L_{\text{active}}(q) \approx \sum_{i \notin S(q)} |P_i| + \sum_{i \in S(q)} |X_i| + |q| + L_{\text{frame}}. \tag{1}$$

Labels, instructions, and chat-template tokens are included in the measured prompt counts, not in the simplified terms of Eq. 1. Keeping unresolved pointers consumes tokens; their causal contribution to answer quality is not established by these experiments.

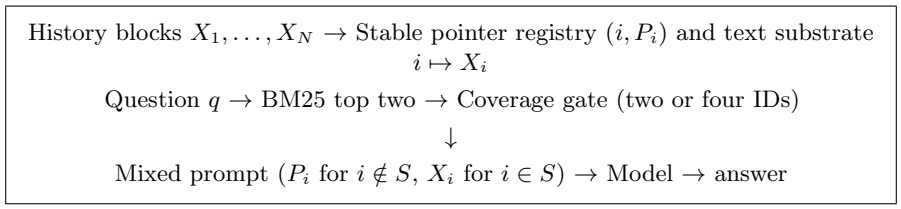


Figure 1. Architecture. Stable pointer IDs bind a compact index to exact text payloads. A lexical router protects two ranked blocks and may request four; the resolver builds a mixed prompt from selected payloads and unresolved pointers. The measured path uses text, not cached KV states.

3.2 Protected lexical routing

BM25 ranks all blocks for q. The first two IDs are protected and form the default candidate set. Let $T(q)$ be the distinct query terms and $T(S)$ the terms appearing in the protected blocks. The router measures unweighted coverage $|T(q) \cap T(S)|/|T(q)|$ and an inverse-document-frequency-weighted coverage

$$C_w(q, S) = \frac{\sum_{t \in T(q) \cap T(S)} \max(0, \mathrm{idf}(t))}{\sum_{t \in T(q)} \max(0, \mathrm{idf}(t))}. \tag{2}$$

The implementation returns zero when the denominator is zero. If $C_w < 0.80$, it requests four candidates. It also expands when $C_w < 0.95$ and the third BM25 score is at least 0.85 times the second; this treats a close ranking boundary as ambiguous. Ties are broken by block order. A 2,560-token cap on resolved text may reject later candidates, so the actual block count can differ from the requested count. Semantic rescue is disabled in all reported runs.

This policy is deliberately inspectable: its route, selected IDs, coverage values, resolved tokens, and selection time are logged per example. It can reduce average prompt length when two blocks suffice, while retaining a four-block path for difficult queries. Coverage is a heuristic over surface terms; paraphrases and multi-hop questions can defeat it.

4 Experimental Protocol

4.1 Tasks and controls

All systems use Qwen2.5-3B-Instruct in NF4 precision on a single 8-GB GPU [12, 4]. RULER uses six selected synthetic tasks, ten seeded examples per task at 4k, 8k, and 16k: three single-needle tasks, one multikey task, variable tracking, and common-word extraction (CWE) [5]. LongBench uses the first 100 examples of each of six English QA tasks, and LoCoMo uses conversations 0–2 [1, 9]. The run fixes the model, tokenizer, instructions, seed, and generation cap within each benchmark: 32 new tokens for RULER and 64 for LongBench and LoCoMo.

The runs compare SemPointer with BM25-4 on all three collections and BM25-2

Table 1. Same-run results. Score is mean [95% example-bootstrap CI] in percent; tokens, answer latency, and TTFT are per-query means. Ingestion is excluded. LoCoMo uses our token-F1 adaptation.

Task	System	n	Score [95% CI]	Tokens	Answer ms	TTFT ms
RULER	SemPointer	180	62.1 [55.5, 68.7]	2,201	1,252	594
RULER	BM25-2	180	59.7 [52.9, 66.6]	1,381	1,051	442
RULER	BM25-4	180	62.3 [55.7, 68.9]	2,416	1,485	849
LongBench	SemPointer	600	10.4 [9.2, 11.7]	1,878	1,732	457
LongBench	BM25-4	600	10.6 [9.4, 11.8]	2,100	1,867	494
LoCoMo	SemPointer	497	6.9 [6.1, 7.7]	1,896	1,849	476
LoCoMo	BM25-4	497	7.2 [6.4, 8.1]	2,122	1,918	519

Table 2. Paired SemPointer minus BM25-4 on identical example IDs. Intervals are 95% bootstrap CIs; p is a two-sided sign-flip test of the score difference. Negative token and latency differences favor SemPointer.

Task	Δ score (pp) [CI]	p	Δ tokens [CI]	Δ answer ms [CI]
RULER	−0.22 [−3.00, +2.67]	0.943	−215 [−311, −117]	−233 [−267, −198]
LongBench	−0.13 [−1.04, +0.79]	0.775	−223 [−277, −167]	−135 [−166, −104]
LoCoMo	−0.37 [−0.85, +0.13]	0.139	−226 [−263, −188]	−68 [−98, −39]

on RULER. We use the earlier paper runs for a same-ID score comparison against the default earlier version and full context. Since these are different executions, we do not infer cross-run latency speedups. All scheduled system-example pairs completed without recorded errors.

The RULER score is case-insensitive answer containment, and selected LongBench tasks use the official normalized QA token-F1 formula. Our selected tasks, prompt construction, and generation caps are not a full official benchmark submission. LoCoMo is scored by the project's normalized token-F1 adaptation, not its official category-specific protocol. A 6.9% LoCoMo result in this paper must not be interpreted as an official benchmark accuracy.

4.2 Statistics and cost accounting

We report per-system mean scores with 95% percentile bootstrap intervals over examples. Paired differences use identical example IDs and 10,000 bootstrap resamples with seed 42; two-sided sign-flip tests describe score differences. These intervals capture example variation within this model and setup, not uncertainty across model families or seeds. Answer latency is timed from the call to answer through its return; ingestion is measured separately and excluded. Time to first token (TTFT) is shown separately. Neither metric is a realistic many-query amortization study, and SemPointer does not claim a measured host-memory or energy reduction.

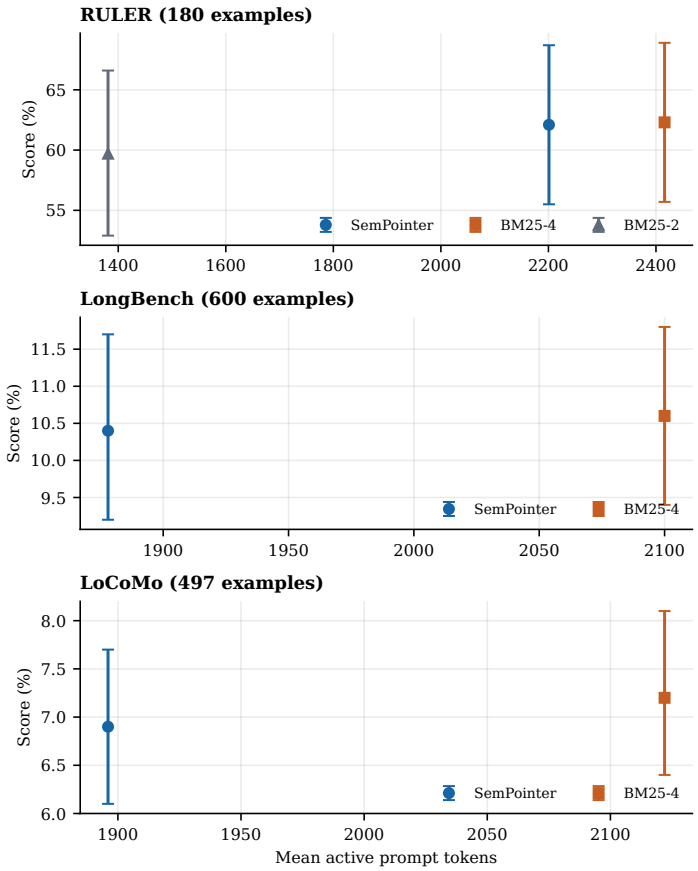


Figure 2. Observed quality–token frontier within each benchmark run. Bars are 95% example-bootstrap score intervals; x coordinates are mean active tokens. Score scales differ by benchmark, and overlapping marginal intervals do not test paired differences.

5 Results

5.1 A quality–cost frontier on the tested runs

Table 1 shows that SemPointer scores 62.1% on RULER, compared with 62.3% for BM25-4. Its mean active prompt is 2,201 rather than 2,416 tokens (8.9% fewer), and its answer latency is 1,252 rather than 1,485 ms (15.7% lower). The paired score difference is -0.22 points, 95% CI $[-3.00, 2.67]$ (Table 2); this does not establish accuracy equivalence. The paired token and latency reductions have intervals wholly below zero. SemPointer versus BM25-2 gains 2.39 score points, CI $[-0.11, 5.11]$, while using more tokens (2,201 versus 1,381) and time (1,252 versus 1,051 ms). It therefore occupies an intermediate point between a cheaper two-block policy and a fixed four-block policy.

On LongBench, SemPointer scores 10.4% versus 10.6% for BM25-4, with 223 fewer tokens and 135 ms lower paired mean latency. On LoCoMo, it scores 6.9% versus 7.2%, with 226 fewer tokens and 68 ms lower paired mean latency. Both score intervals span zero difference. The score floors are low, so the claim is about reduced cost at the measured score level, not strong task performance.

The default earlier version scores 45.6% on the same 180 RULER IDs. SemPointer scores 16.56 points higher, paired CI [11.00, 22.56], $p < 0.001$. Full context from that earlier run scores 69.8%; SemPointer remains 7.67 points lower, paired CI [−12.11, −3.44]. The redesign improves the implementation substantially, yet it does not close the full-context accuracy gap.

Figure 2 makes the budget choice visible. RULER's BM25-2 point is cheaper but lower scoring; SemPointer spends more tokens to approach the BM25-4 point. The LongBench and LoCoMo panels show smaller score ranges with a consistent leftward movement in active tokens. Because these are mean positions, the plot cannot show whether individual questions benefit; the paired contrasts in Fig. 3 and Table 2 carry that inference.

Latency accounting. The SemPointer answer call includes selection, prompt assembly, and generation. The measured TTFT is 594 versus 849 ms for BM25-4 on RULER, 457 versus 494 ms on LongBench, and 476 versus 519 ms on LoCoMo. Selection itself averages 2.2, 2.8, and 2.0 ms for SemPointer, compared with 1.3, 0.2, and 0.2 ms for BM25-4. The router therefore adds a small selection cost while the answer call becomes faster overall. Fewer active tokens are a plausible contributor to the TTFT reduction, but these measurements do not isolate kernel-level causes. Every benchmark example is ingested separately in this runner; the figures do not establish how either system would behave with shared prefill caches, batched requests, or repeated questions over one stored history.

5.2 Where routing helps and fails

The router takes its two-block route on 118/180 RULER examples, 421/600 LongBench examples, and 409/497 LoCoMo examples (Fig. 4). RULER multikey retrieval is 93.3% for SemPointer and BM25-4, while the second single-needle variant rises from 80.0% for BM25-4 to 86.7% for SemPointer. Variable tracking moves in the other direction: 28.7% versus 36.7%. These task values each summarize only 30 examples and should not be treated as stable per-task rankings.

CWE remains at 0.7% for SemPointer, BM25-4, the earlier version, and full context under the 32-token cap. Its target contains ten words, and observed outputs can be truncated before listing them. This is a protocol floor requiring an output-length audit before method comparisons. The weighted-coverage gate also measures lexical overlap, so a route decision can be overconfident for paraphrases or multi-step evidence. Conditional scores by route are observational: hard and easy questions are not randomly assigned to the two paths.

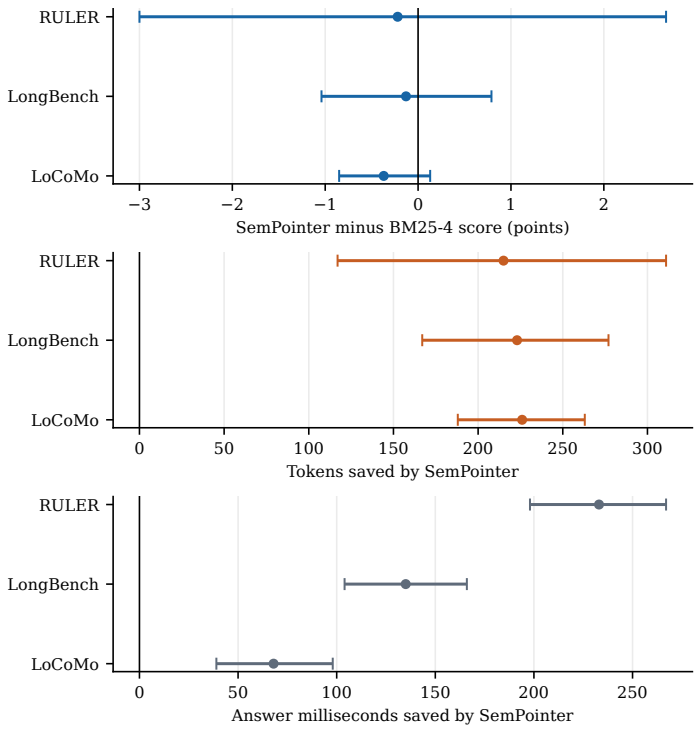


Figure 3. Paired SemPointer-minus-BM25-4 score differences and BM25-4-minus-SemPointer token and answer-latency savings. Points are means; bars are 95% paired bootstrap intervals over shared example IDs. All three score intervals include zero, while the measured cost-savings intervals do not. Ingestion is excluded from latency.

6 Discussion and Limitations

What the evidence supports. The registry is useful as a budgeted serving interface: it retains stable addresses, logs which evidence was resolved, and conditionally exposes more text. On the evaluated model and subsets, the adaptive policy reduces average active tokens and same-run latency against fixed BM25-4 without a measured large change in mean answer score. The larger gain over the earlier version is consistent with replacing a harmful attention-fused selector by a lexical-first router.

What remains unproven. The current suite has no pointer-off control under the same adaptive route, so it cannot assign the efficiency gain specifically to readable pointer content rather than the routing policy. It has no strict per-example iso-token BM25 baseline, no held-out seed run, and no semantic-rescue ablation. The RULER holdout and rescue stages in the runner were not present among the completed result files. Hyperparameters were not selected on an

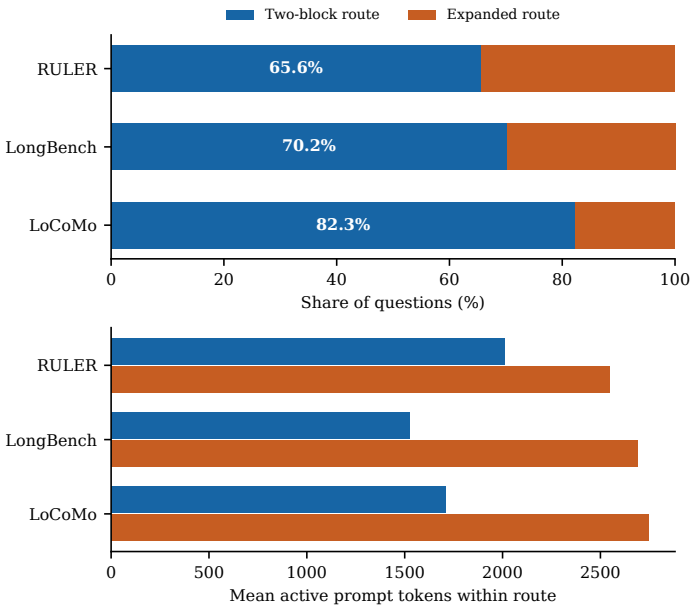


Figure 4. Observed router decisions and active-token cost conditional on the chosen route. The upper bars partition the questions; the lower bars describe the mean prompt size after each route (blue: two-block route, orange: expanded route). Route-conditioned question sets differ in difficulty, so their scores would not be a causal expansion test.

independent development split, making generalization uncertain. The LongBench and LoCoMo scores use restricted subsets and adapted protocol details. Results come from one quantized model on one GPU; elapsed times need replication under serving concurrency and repeated queries. The text path does not validate the separate KV substrate of the earlier version.

Deployment interpretation. An application with a strict prompt budget and inspectable evidence routing may benefit from this design. If maximum answer accuracy is the sole goal, full context remains stronger on RULER. If minimum tokens or latency is the sole goal, BM25-2 is cheaper in the tested RULER run. A deployment decision should choose a point on that measured quality–cost frontier, then verify it on its own query distribution.

What a stronger evaluation would resolve. A causal test should run the identical router with pointer entries removed, then compare scores at the same resolved blocks and active-token budget. A held-out seed should fix the coverage thresholds before measuring the final task set. Longer output caps are especially important for CWE and for LongBench tasks whose official generation limits exceed this run's shared cap. Repeated-query traces would let us measure

registry construction once and charge it across many answers; under the present one-example ingestion loop, the savings in Fig. 3 are per answer after ingestion. These controls would show whether the address layer adds value beyond adaptive BM25 passage selection and whether the present speed ranking survives realistic serving workloads.

7 Conclusion

SemPointer turns a fixed block selection into an inspectable two-or-four-block routing decision. In completed runs, this repairs much of the earlier version's RULER accuracy loss and reduces mean tokens and answer latency relative to fixed BM25-4 at similar observed scores across three tested collections. The contribution is adaptive, addressable context under a modest hardware budget. Claims of pointer-specific causal value, accuracy equivalence, official benchmark standing, and cross-model robustness await targeted controls and held-out evaluation.

References

[1] Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., Du, Z., Liu, X., Zeng, A., Hou, L., Dong, Y., Tang, J., Li, J.: LongBench: A bilingual, multitask benchmark for long context understanding. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL). pp. 3119–3137 (2024)

[2] Cheng, X., Wang, X., Zhang, X., Ge, T., Chen, S.Q., Wei, F., Zhang, H., Zhao, D.: xRAG: Extreme context compression for retrieval-augmented generation with one token. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 37 (2024)

[3] Cormack, G.V., Clarke, C.L.A., Büttcher, S.: Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In: Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval. pp. 758–759 (2009). https://doi.org/10.1145/1571941.1572114

[4] Dettmers, T., Pagnoni, A., Holtzman, A., Zettlemoyer, L.: QLoRA: Efficient finetuning of quantized LLMs. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 36 (2023)

[5] Hsieh, C.P., Sun, S., Kriman, S., Acharya, S., Rekesh, D., Jia, F., Zhang, Y., Ginsburg, B.: RULER: What's the real context size of your long-context language models? In: Conference on Language Modeling (COLM) (2024)

[6] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., tau Yih, W., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 33 (2020)

[7] Li, Y., Huang, Y., Yang, B., Venkitesh, B., Locatelli, A., Ye, H., Cai, T., Lewis, P., Chen, D.: SnapKV: LLM knows what you are looking for before generation. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 37 (2024)

[8] Lu, S., Wang, H., Rong, Y., Chen, Z., Tang, Y.: TurboRAG: Accelerating retrieval-augmented generation with precomputed KV caches for chunked text. arXiv preprint arXiv:2410.07590 (2024)

[9] Maharana, A., Lee, D.H., Tulyakov, S., Bansal, M., Barbieri, F., Fang, Y.: Evaluating very long-term conversational memory of LLM agents. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL). pp. 13851–13870 (2024)

[10] Mohtashami, A., Jaggi, M.: Landmark attention: Random-access infinite context length for transformers. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 36 (2023)

[11] Mu, J., Li, X.L., Goodman, N.: Learning to compress prompts with gist tokens. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 36 (2023)

[12] Qwen Team: Qwen2.5 technical report. arXiv preprint arXiv:2412.15115 (2024)

[13] Reimers, N., Gurevych, I.: Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP). pp. 3982–3992 (2019)

[14] Robertson, S., Zaragoza, H.: The probabilistic relevance framework: BM25 and beyond. Foundations and Trends in Information Retrieval 3(4), 333–389 (2009)